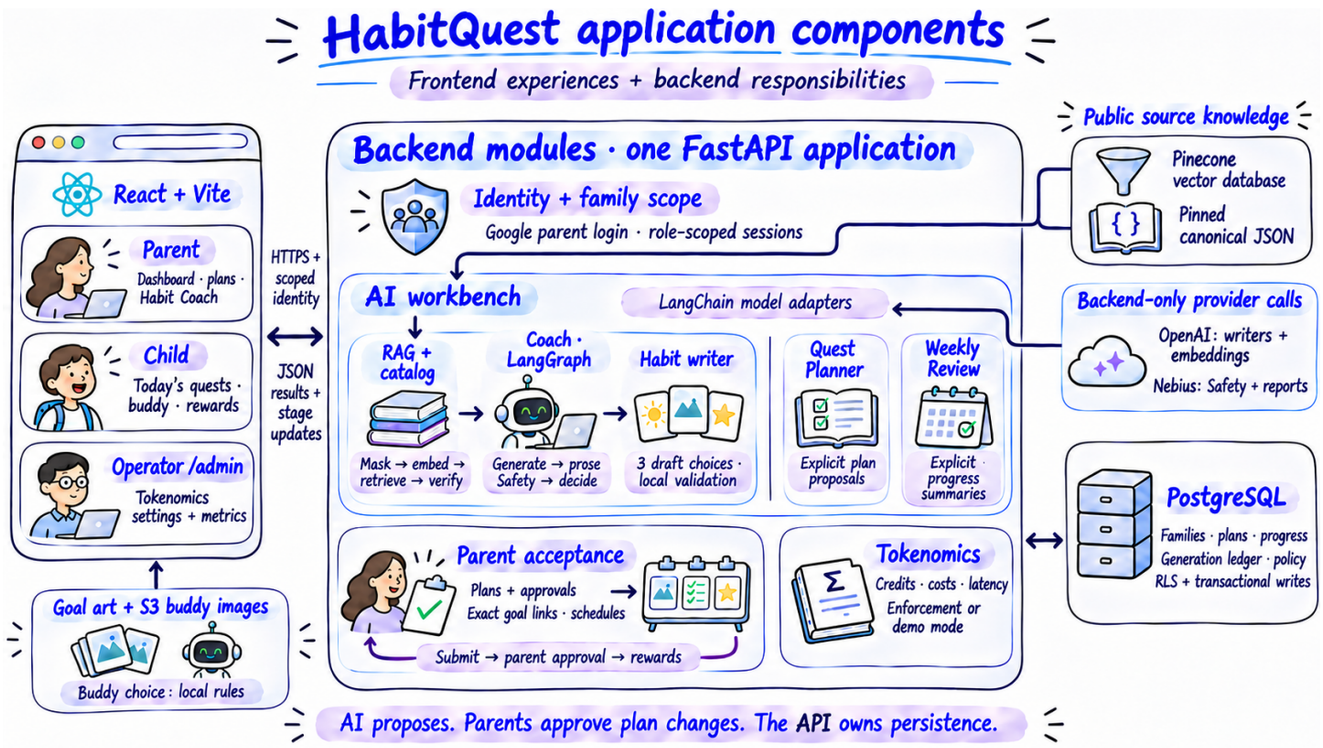
HabitQuest

HabitQuest helps families turn researched goals into small, achievable habits and daily quests. Parents choose and adapt a child's plan, children follow picture-based quests with an adventure buddy, and parent-approved effort becomes progress and rewards. Optional AI helpers support goal discovery, habit choices, plan adjustments and weekly reflection, while parents stay in control of what is accepted.

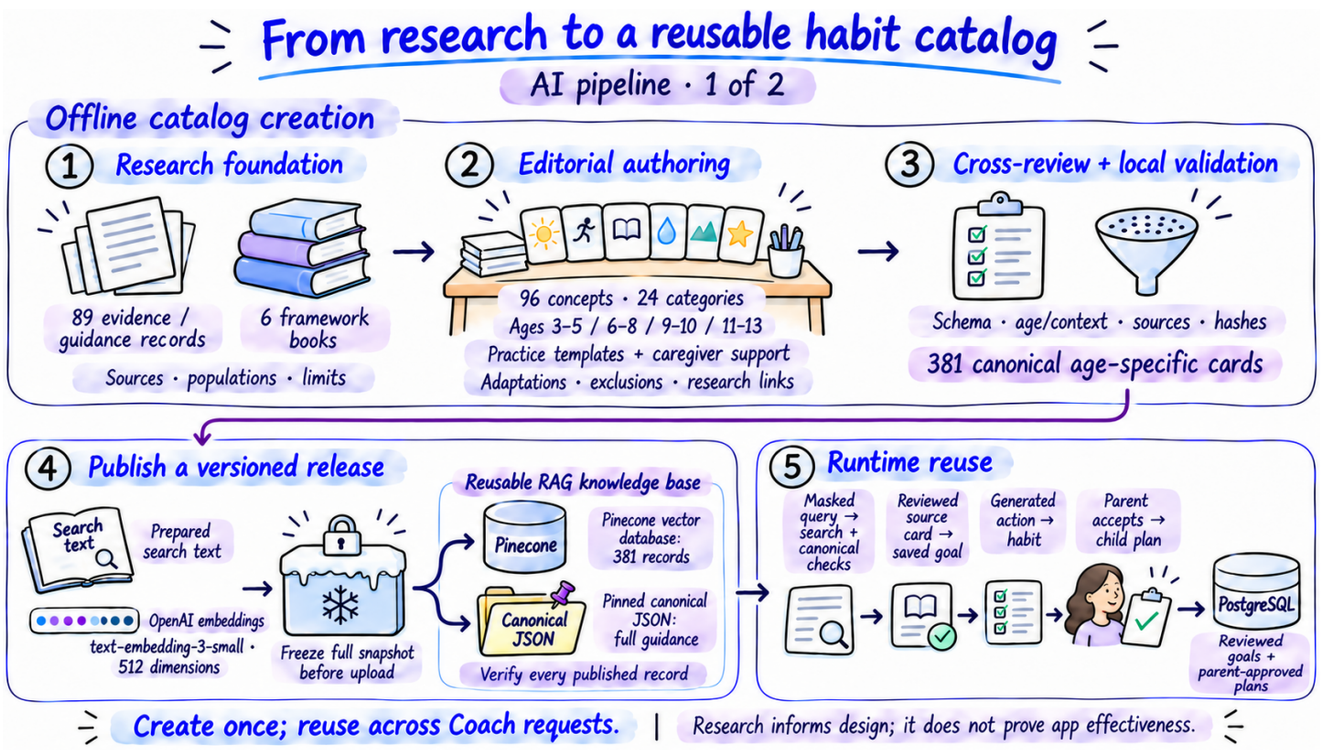
01 Components	02 Research	03 AI runtime	04 Coach	05 Planner
06 Review	07 Safety	08 Tokenomics	09 Evaluations	10 AWS

01 Application components



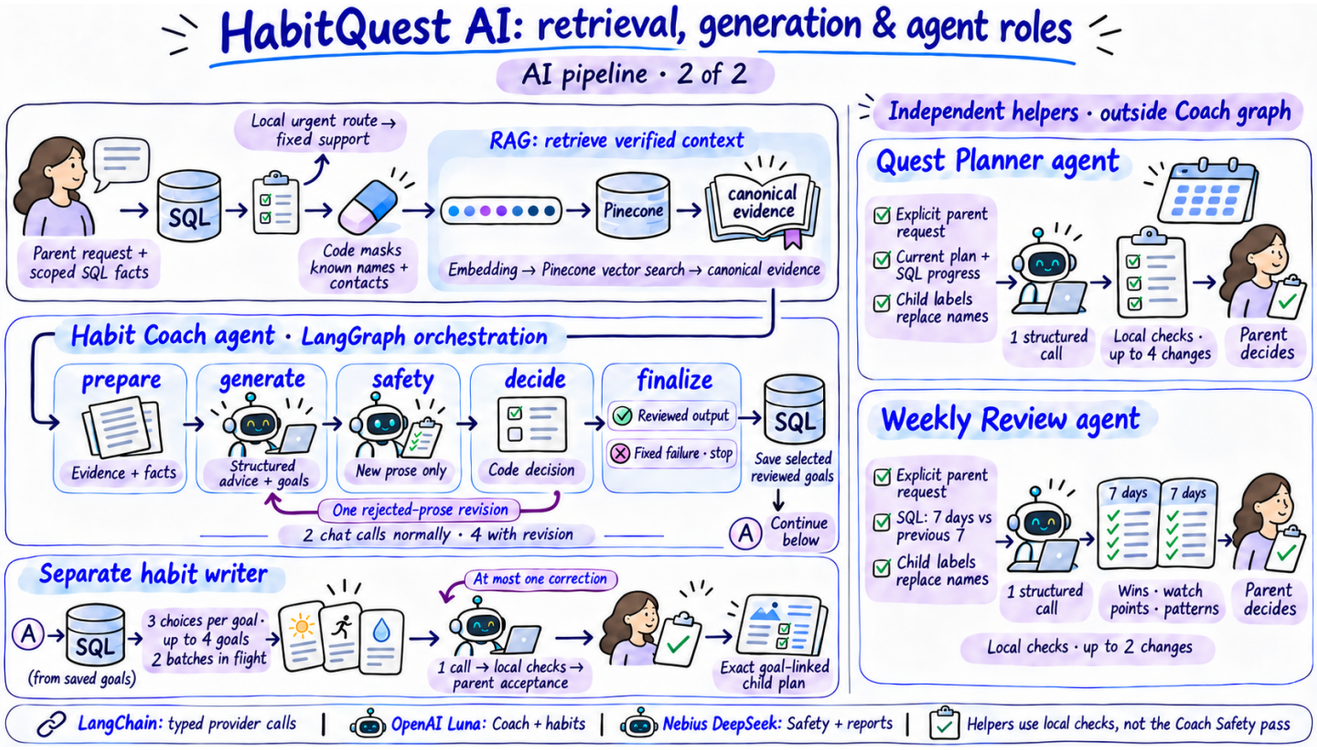
The frontend presents parent, child and operator experiences. One modular FastAPI backend owns authorization, retrieval, bounded AI calls, exact goal links, plans, approval rules and accounting. LangGraph belongs to the Coach module; these responsibilities are not separate microservices.

02 AI pipeline · 1 of 2: research and catalog



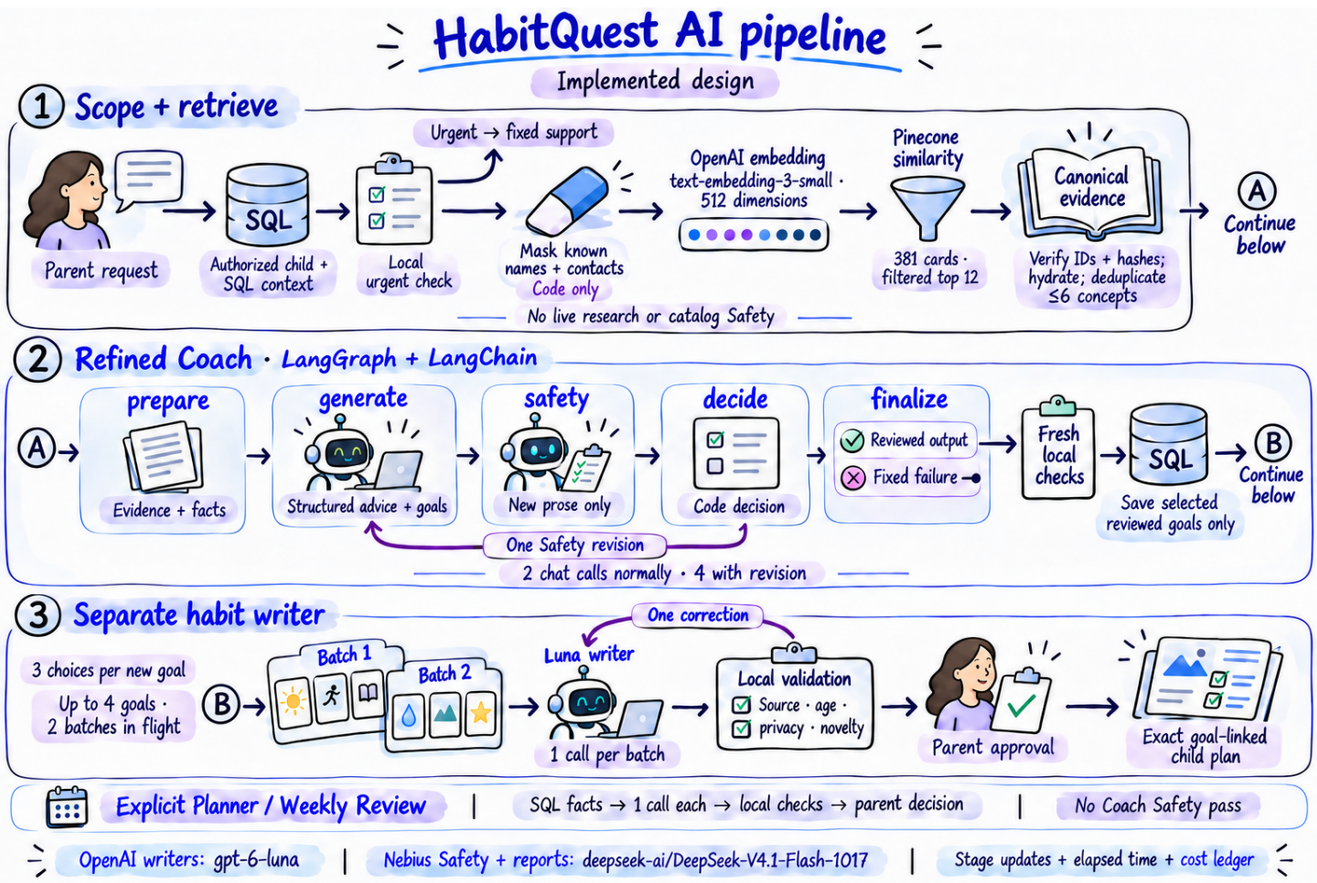
Offline research, editorial authoring and cross-review create reusable age-specific practice cards. Prepared search text is embedded, frozen and published to Pinecone; full guidance remains in pinned canonical JSON. Live requests reuse the release. Reviewed source cards become goals, while personalized habit choices are generated later.

03 AI pipeline · 2 of 2: runtime and agents



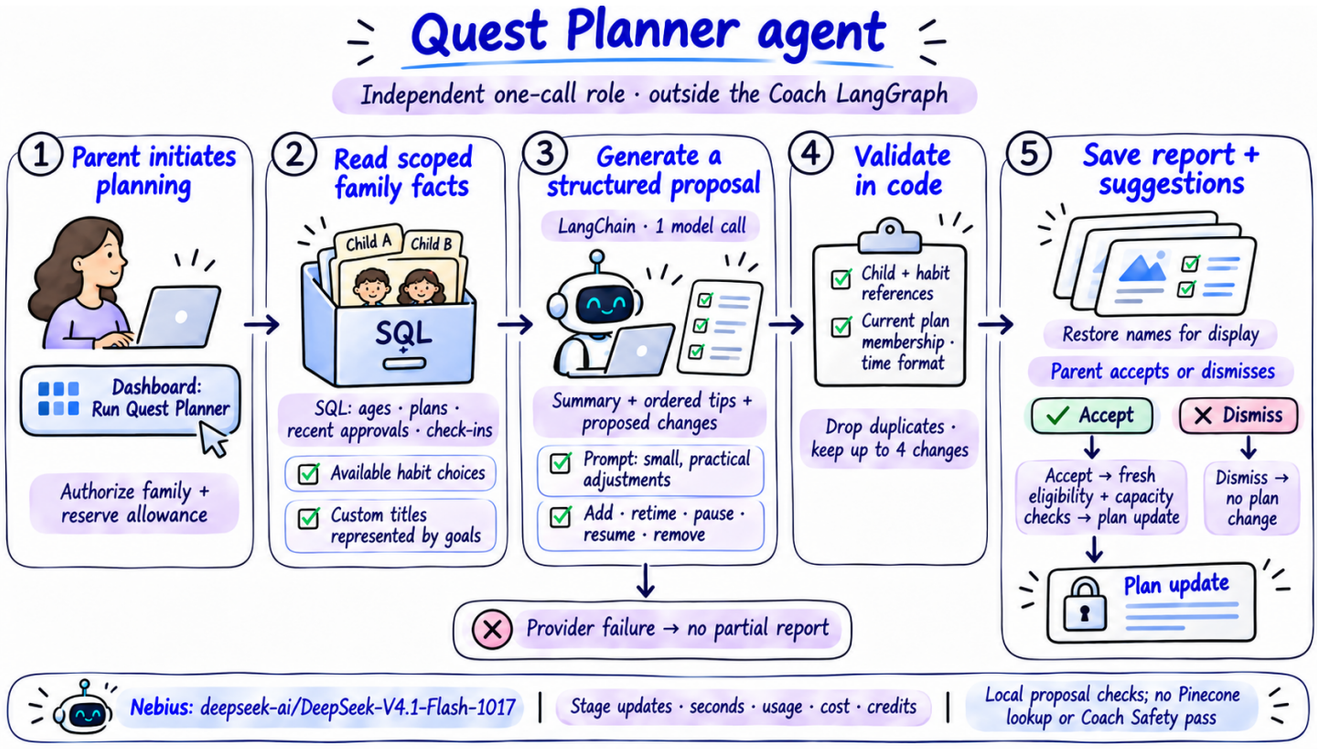
RAG supplies verified context to the five-node Coach graph. New prose receives model Safety; separate writer batches prepare habit choices. Quest Planner and Weekly Review are independent, explicitly triggered agent roles outside that graph. Each helper is a structured model call plus local checks, not an autonomous multi-agent loop.

04 Habit Coach: detailed runtime pipeline



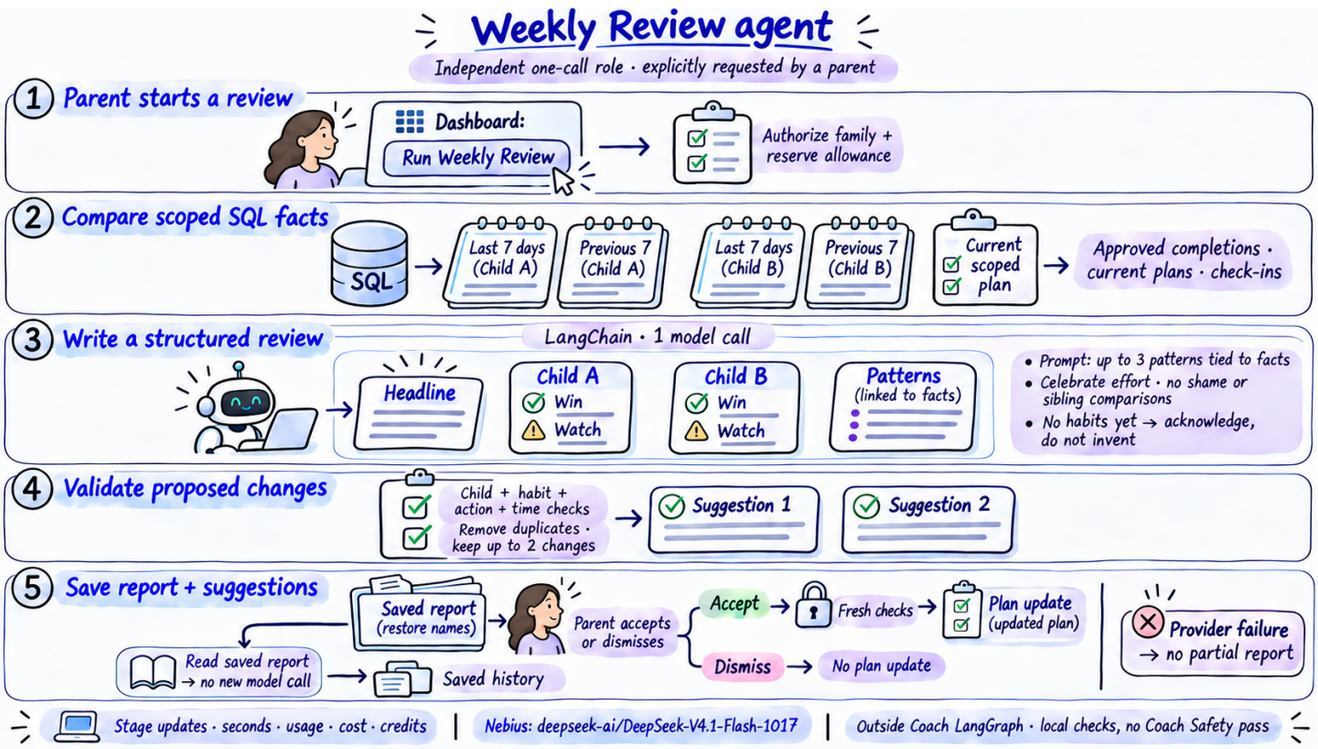
Preserved original runtime illustration: it explains the API request-to-result pipeline without offline catalog creation. Follow A from verified retrieval into the Coach graph and B from reviewed saved goals into separate habit batches. Planner and Review retain their own dedicated slides.

05 Quest Planner agent



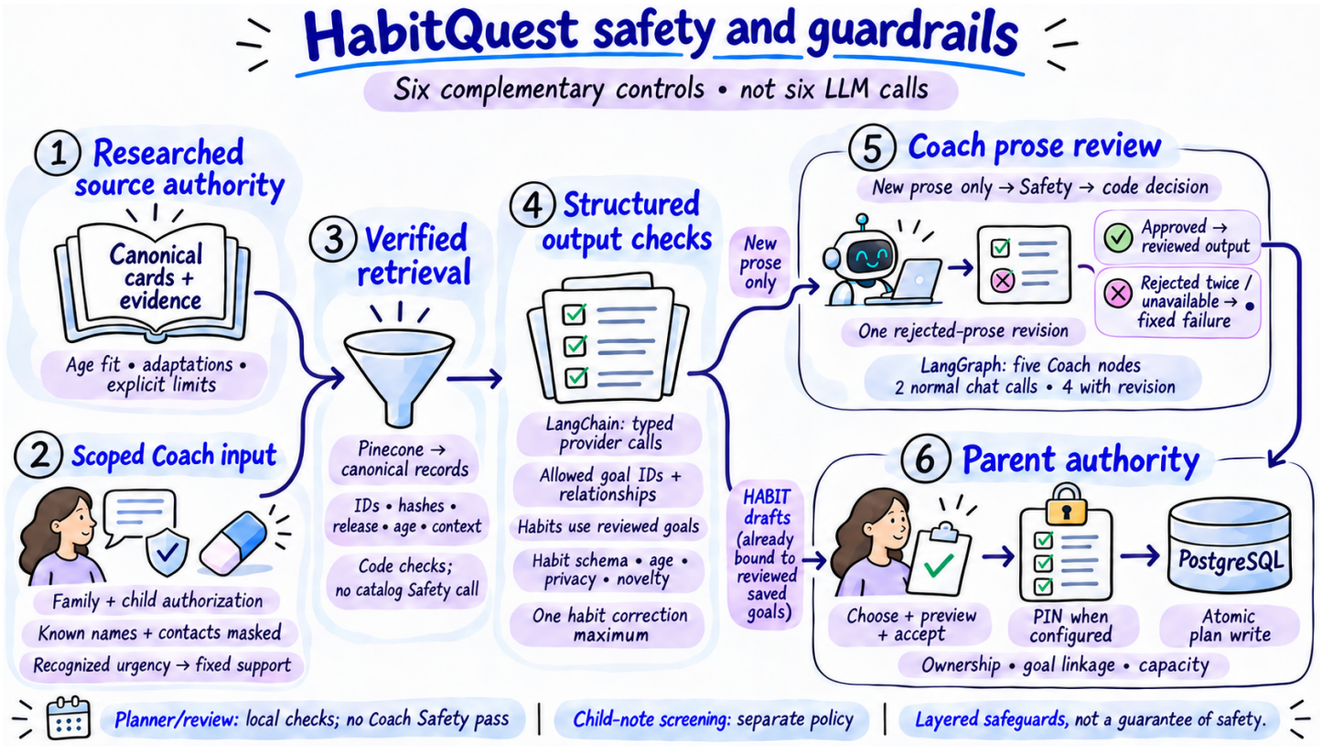
Quest Planner focuses on practical adjustments to the family plan. One structured call uses scoped SQL facts; code checks references and retains up to four suggestions. The parent decides, with fresh eligibility and capacity checks before changes are applied. It runs outside the Coach graph.

06 Weekly Review agent



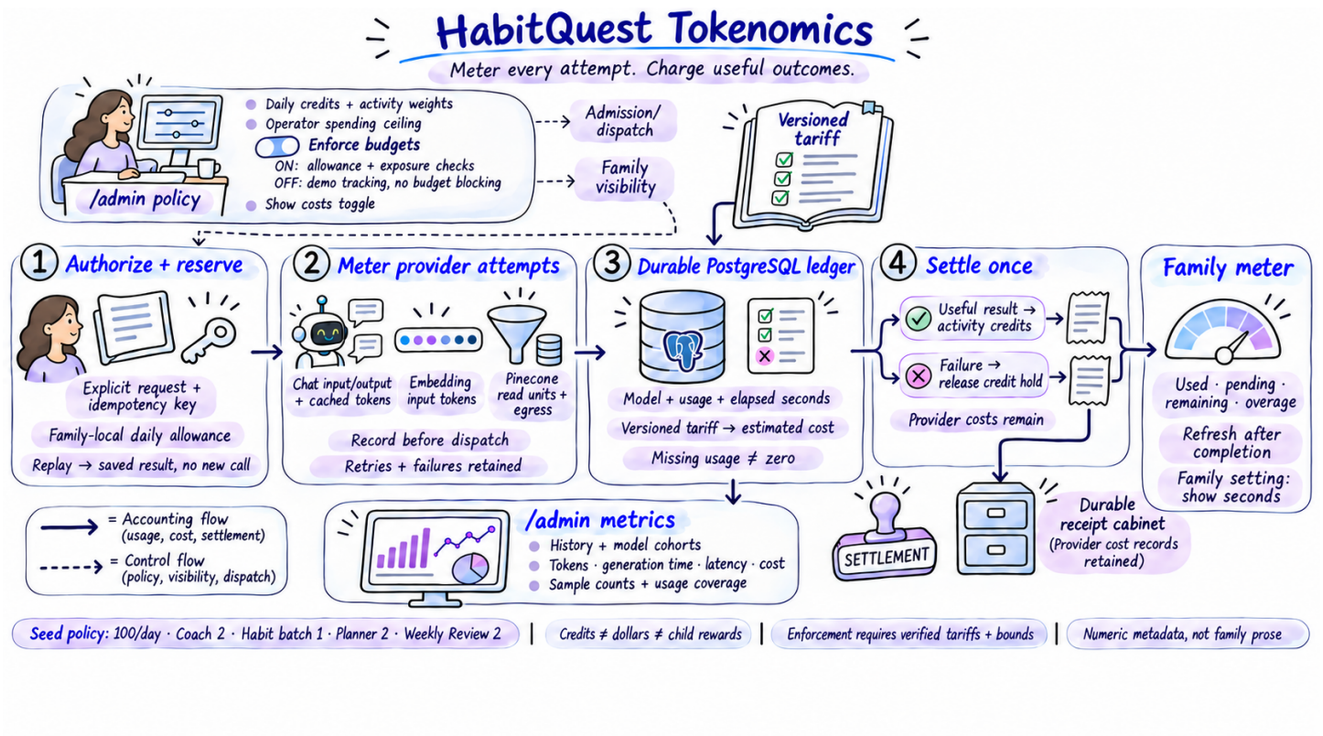
Weekly Review interprets recent progress rather than creating a schedule automatically. It compares two seven-day SQL windows, produces a parent-facing report and at most two locally checked suggestions. Parents decide on changes; reading the saved report makes no new model call.

07 Safety guardrails



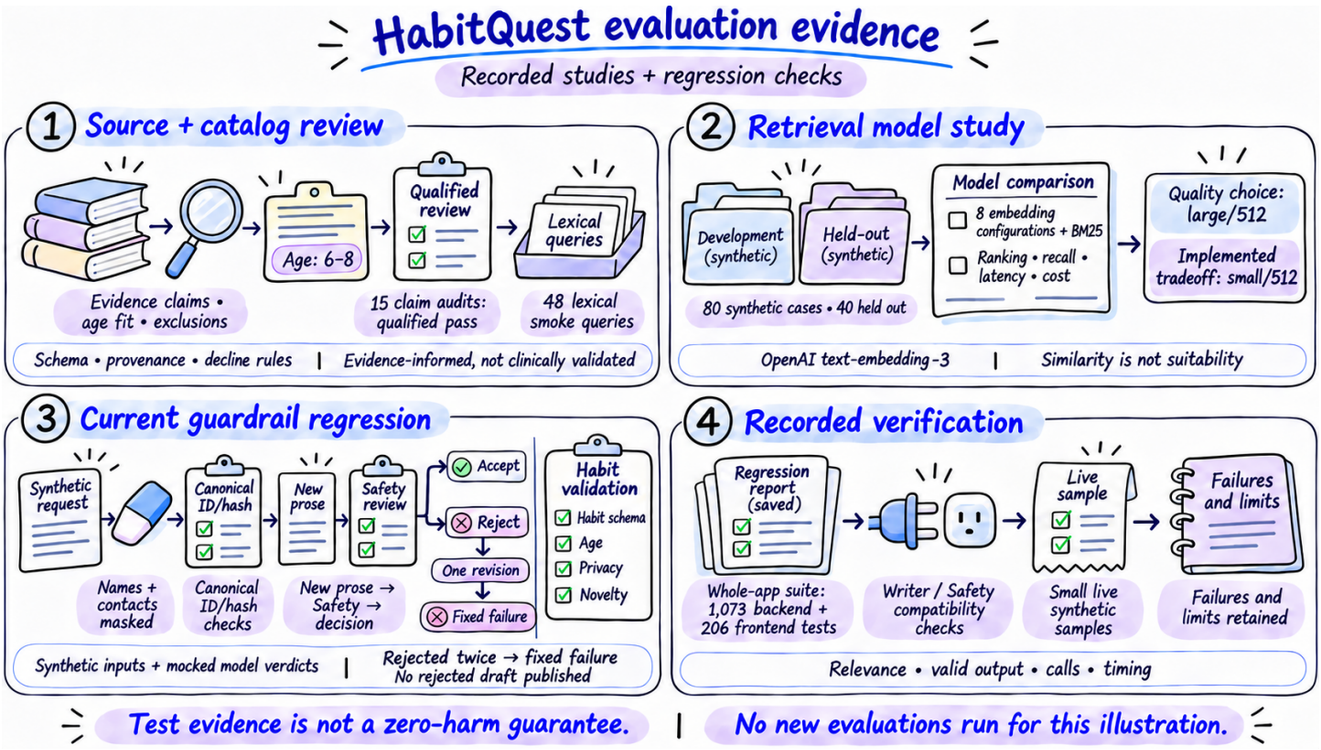
Six complementary controls cover researched source authority, scoped Coach input, verified retrieval, structured output checks, Coach prose review and parent authority. They are not six LLM calls. Trusted catalog lookup and bounded habit checks do not receive an extra whole-content Safety pass.

08 Tokenomics and monitoring



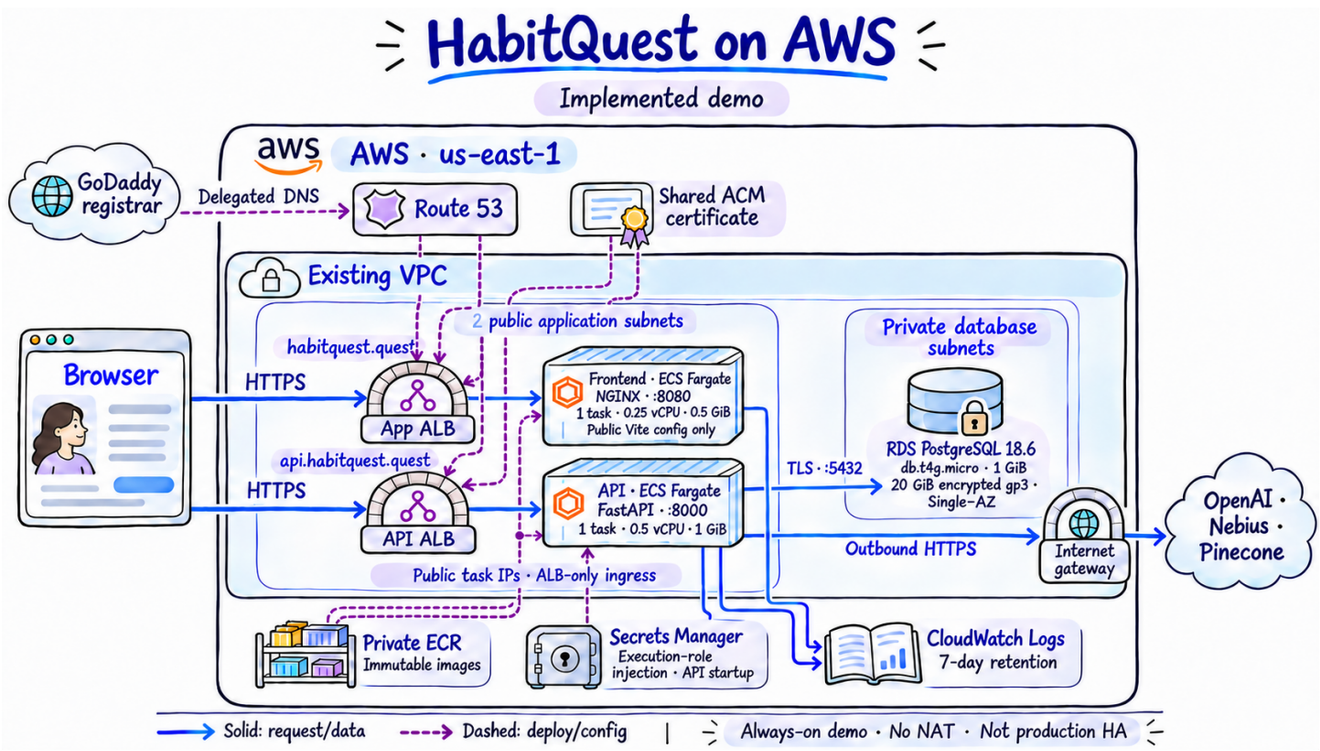
Every provider attempt retains usage, timing and estimated cost, including embeddings, search, retries and failures. Useful results settle family credits once; failed work releases credit holds without erasing provider cost. Admin can enforce allowance and exposure gates or track demo overage. Enforcement requires verified tariffs and bounded exposure.

09 Evaluation evidence



Recorded source audits, a synthetic embedding study, mocked guardrail regressions and small live-provider samples answer different questions. They support content, model and control-flow decisions. They do not establish a zero-harm guarantee, clinical effectiveness or a new production latency benchmark.

10 AWS deployment



The browser independently reaches the app and API through two public ALBs. Public-subnet Fargate tasks use restricted ALB-only ingress and internet-gateway egress; RDS stays private. ECR supplies both services, API secrets arrive at startup and CloudWatch retains logs. This always-on demo uses no NAT and is not production HA.